

29 | 弄清现状：新架构预览版究竟长什么样？

2022-10-1 蒋宏伟



你好，我是蒋宏伟。

上一节课，我们使用了三招，也就是“时光机”、“找线头”和“鸟瞰图”，初步了解了 React Native 第一版的老架构长什么样。我们学习第一版 React Native 架构的目标不是为了了解过去，而是为了搞清楚 React Native 新架构的现状，以及新架构未来会去向何方。

因此，今天我们的目标就是承接上一节课讲过的方法，画一张新架构的“鸟瞰图”，看看当前 0.70 版本的新架构预览版究竟是怎么设计的。

在这节课的学习过程中，我们会涉及部分的源码，所以我强烈建议你打开 Xcode 或者 Android Studio 对源码部分进行断点调试，这样才能学得更透彻。

读文章

不过，为了帮你理清楚 React Native 团队的架构设计思路，我想先带你看看官方是怎么[介绍新架构](#)的。

在官方对新架构介绍的文章中，你可以看到官方重点提到了新架构的三大件，分别是**Turbo Modules** 模块系统、**Fabric** 组件系统和**Codegen** 代码生成工具。

Turbo Modules 模块系统是全新的原生模块系统，代替的是老架构中的 Native Modules。官方文档中的 API，比如 Animated、Platform、Keyboard，在老架构中都是采用 Native Modules 来实现的，在新架构中都改用 Turbo Modules 进行实现了。

Fabric 组件系统呢，是一套全新的原生组件系统，代替的是老架构中的 Native Components。在官方文档中的 Component，比如 View、Image、ScrollView，在老架构都是采用 Native Components 实现的，在新架构中都改用 Fabric Components 进行实现了。

至于Codegen 代码生成工具，它的作用是在编译时把 TypeScript 写的类型声明代码，转换为用 C++ 写的类型声明代码，供给原生模块使用。虽然Codegen 是个非常有意思的工具，但它是在编译时工作的，而我们今天要画的“鸟瞰图”是新架构运行时的整体逻辑。

因此，这节课我们就不再对 Codegen 进行展开讲解了，我们把研究的重点放在 Turbo Modules 和 Fabric Components 上。

找线头

那么，新架构中的 Turbo Modules 和 Fabric Components 的代码在哪呢？找到代码是画“鸟瞰图”的关键步骤，这里我们运用上节课讲过的“找线头”的方法来试试。

正如我们上节课所说，React Native 0.70 版比 React Native 的第一个版本的代码量大 10 倍。第一版文件数量只有 344 个，到了 0.70 版本就变成了 3,840 个了。那怎么在 3,840 个文件中，弄明白 Turbo Modules 和 Fabric Components 是如何初始化的，又是如何运作的呢？

我们就得找找线头了。

首先，你可能会想，在老架构中，我们沿着 main 函数找到的 AppDelegate.m 文件，是否还能作为弄清新架构的关键线头呢？

我们来看代码，AppDelegate.m 文件的 didFinishLaunchingWithOptions 回调，该回调会在程序启动完成时执行 React Native 的初始化，它代码摘要如下：

```
- (BOOL)application:(UIApplication *)application didFinishLaunchingWithOptions:(NSDictionary
{
    // 初始化 bridge
```

```

RCTBridge *bridge = [[RCTBridge alloc] initWithDelegate:self launchOptions:launchOptions];

#if RCT_NEW_ARCH_ENABLED
    // 如果是开启了新架构，那么生成 bridgeAdapter 对 bridge 进行适配。
    _bridgeAdapter = [[RCTSurfacePresenterBridgeAdapter alloc] initWithBridge:bridge contextCo
    bridge.surfacePresenter = _bridgeAdapter.surfacePresenter;
#endif

    // 创建根视图
    UIView *rootView = RCTAppSetupDefaultRootView(bridge, @"RN70", initProps);

    // 将根视图挂载到 window 上
    self.window.rootViewController = rootView;
}

```

从上述代码摘要中，你可以看出在 `didFinishLaunchingWithOptions` 回调中，新架构和老架构做的事情差不多。这段代码先是初始化了 `Bridge`，然后又生成了一个 `bridgeAdapter`，你看这个变量命名，就知道它是新架构为兼容老架构做的适配处理，最后就是创建 `React Native` 的根视图，并将根视图挂载到 `window` 上和进行页面展示的过程了。

你可以看到，在 `didFinishLaunchingWithOptions` 回调中，根本没有新架构的 `Turbo Modules` 和 `Fabric Components` 的相关代码，反倒是初始化了老架构的 `Bridge`。

源码看到第一步，很多同学可能就糊涂了，新架构中竟然没有新架构代码，只有老架构的 `Bridge`，这是怎么回事呢？

有些认真的同学可能还会通过 `Xcode` 断点一层层地跟下去，但断点跟下去大概率还是找不到 `Turbo Modules` 和 `Fabric Components` 是如何初始化的，能找到还是 `Bridge` 初始化的相关代码。

那怎么办呢？条条大路通罗马，此路不通有路通啊。

这是因为新架构采用了多线程的架构，单纯的断点跟下去，执行过程会反复地在 `Main` 线程和 `JavaScript` 线程横跳，让人摸不着北。既然 `AppDelegate.m` 这根线头解不开，那么咱们就想办法换根其他线头。

连接点

新的线头就是连接点。

所谓的连接点就是 Native 和 JavaScript 通讯的关键对象或方法。

上一讲中，我们提到过 Native 之所以能执行 JavaScript Bundle 文件，就是因为 Native 能调用 JavaScript Engine 的能力。所谓的 Bridge 也好，Turbo Modules、Fabric Components 也罢，无非就是 Native 能够调用 JavaScript，而 JavaScript 也能调用 Native，二者通过一些关键的连接点实现了双向调用。

因为 Native 能够控制 JavaScript Engine，所以 Native 能够操作所有的 JavaScript 变量，也能调用任意的 JavaScript 函数。因此这里关键的难点是，JavaScript 怎么调用 Native 函数呢？

这就要依赖 Native 向 JavaScript 上下文中注入的特殊变量了。在 React Native 新架构中，Native 会向 JavaScript 中注入三个关键变量，这三个变量就是连接点了，它们分别是：

nativeFlushQueueImmediate，它是 Bridge 部分中 JavaScript 调用 Native 的连接点；

__turboModuleProxy，它是 Turbo Modules 部分中 JavaScript 调用 Native 的连接点；

nativeFabricUIManager，它是 Fabric Components 部分中 JavaScript 调用 Native 的连接点。

有了以上三个连接点，就能快速圈定我们读源码的内容，帮我们快速理解源码的实现逻辑。你可以使用 Xcode 搜索三个连接点的关键字，这样你就能在 3,840 文件中，快速定位到 6 个关键文件了，包括 3 个连接点分别在 3 个 Native 文件中初始化的实现代码，以及 3 个连接点在 3 个 JavaScript 文件中是如何使用的代码。

接下来，我们就分别对这三个连接点要实现的 Bridge、Turbo 和 Fabric 三种通信方式进行详细的代码剖析。

Bridge

先来看 Bridge 部分的相关代码和调用时序图：

```
// ReactCommon/jsiexecutor/jsireact/JSIExecutor.cpp
// 在 ios 初始化 Bridge 的连接点
runtime_>global().setProperty(
    *runtime_,
    "nativeFlushQueueImmediate",
```

```

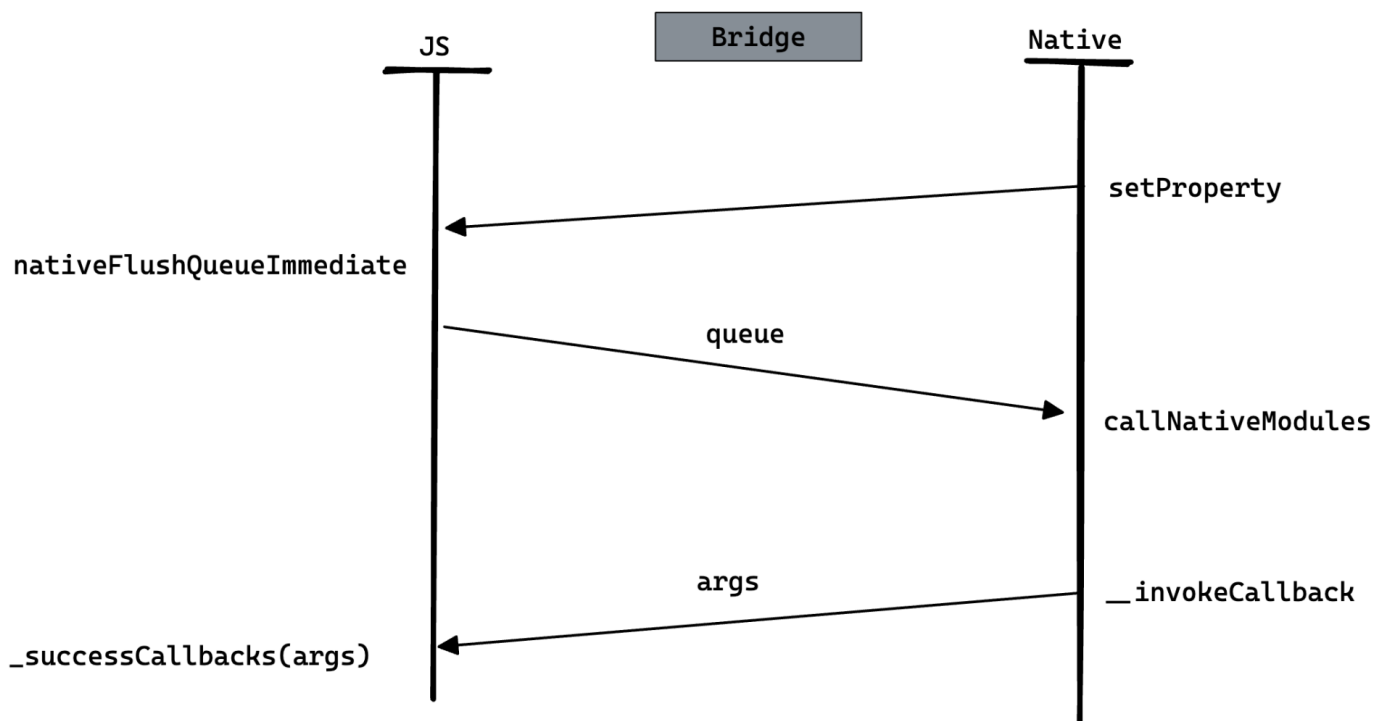
Function::createFromHostFunction(
    *runtime_,
    PropNameID::forAscii(*runtime_, "nativeFlushQueueImmediate"),
    1,
    [this](
        jsi::Runtime &,
        const jsi::Value &,
        const jsi::Value *args,
        size_t count) {
        callNativeModules(args[0], false);
        return Value::undefined();
    }));

```

```

// 在 js 侧使用 Bridge
// queue = '[[15],[0],[[35,500,1664101345473,false]],175]'
nativeFlushQueueImmediate(queue)

```



首先，在 C++ 文件 JSIExecutor.cpp 中，React Native 先通过 `runtime_>global()` 获取了 JavaScript 上下文中的 `global` 变量，然后再通过 `setProperty` 方法，在 JavaScript 的 `global` 的 `nativeFlushQueueImmediate` 属性上，把 Native 的函数 `callNativeModules()` 给连接起来了，`callNativeModules` 就是 Native 中，调用原生模块的实现函数。

这样，当我们在 JavaScript 侧调用 `nativeFlushQueueImmediate` 函数时，会将其参数 `queue` 传给 `callNativeModules` 处理。比如，二维数组 `queue` 中的第一项 `[[15]]`，就代表着批量调用 15 对应的原生 Timing 模块。

当我们提到在 JavaScript 使用 Bridge 时，绝大多数情况指的就是在 JavaScript 中调用 nativeFlushQueueImmediate 函数。

但是，nativeFlushQueueImmediate 函数和我们平时调用的普通函数不一样。普通函数有入参，有执行过程，还有会执行返回值，而 nativeFlushQueueImmediate 是一个批处理函数，它的入参是个二维数组，二维数组中的内容，记录了多个普通函数的名字、入参，然后 Native 再根据普通函数的名字、入参分别对它们进行调用。

那这些在 Native 中调用的普通函数的返回值如何返回给 JavaScript 呢？

是继续通过 nativeFlushQueueImmediate 的 return 值把所有普通函数的执行结果批量返回吗？批量返回肯定不靠谱的，这样一来，只要有一个普通函数执行慢，就会把整个队列给卡死，因此 nativeFlushQueueImmediate return 的是 undefined。

批量调用的普通函数的返回值，是通过其他的 Native 途径一个一个返回给 JavaScript 的。Native 会直接调用 JavaScript 中的 invokeCallback 方法，用最快的速度将普通函数的执行结果给分别返回。

以上，就是 Bridge 实现 JavaScript 与 Native 双向操作的核心原理。

目前，在 0.70 版本的新架构预览版中，还保留了部分的 Bridge 调用，但是一些核心的操作，比如获取 Platform 信息、创建 React Native 节点等，这些操作都已经采用 Turbo Modules 和 Fabric Components 实现了。

Turbo Modules

接着，我们再来看看 Turbo Modules 的底层实现，它才是新架构的核心模块。Turbo Modules 连接点的实现代码和调用时序图如下：

```
// ReactCommon/react/nativemodule/core/ReactCommon/TurboModuleBinding.cpp
// 在 ios 初始化 Turbo Modules 的连接点
runtime.global().setProperty(
    runtime,
    "__turboModuleProxy",
    jsi::Function::createFromHostFunction(
        runtime,
        jsi::PropNameID::forAscii(runtime, "__turboModuleProxy"),
```

```

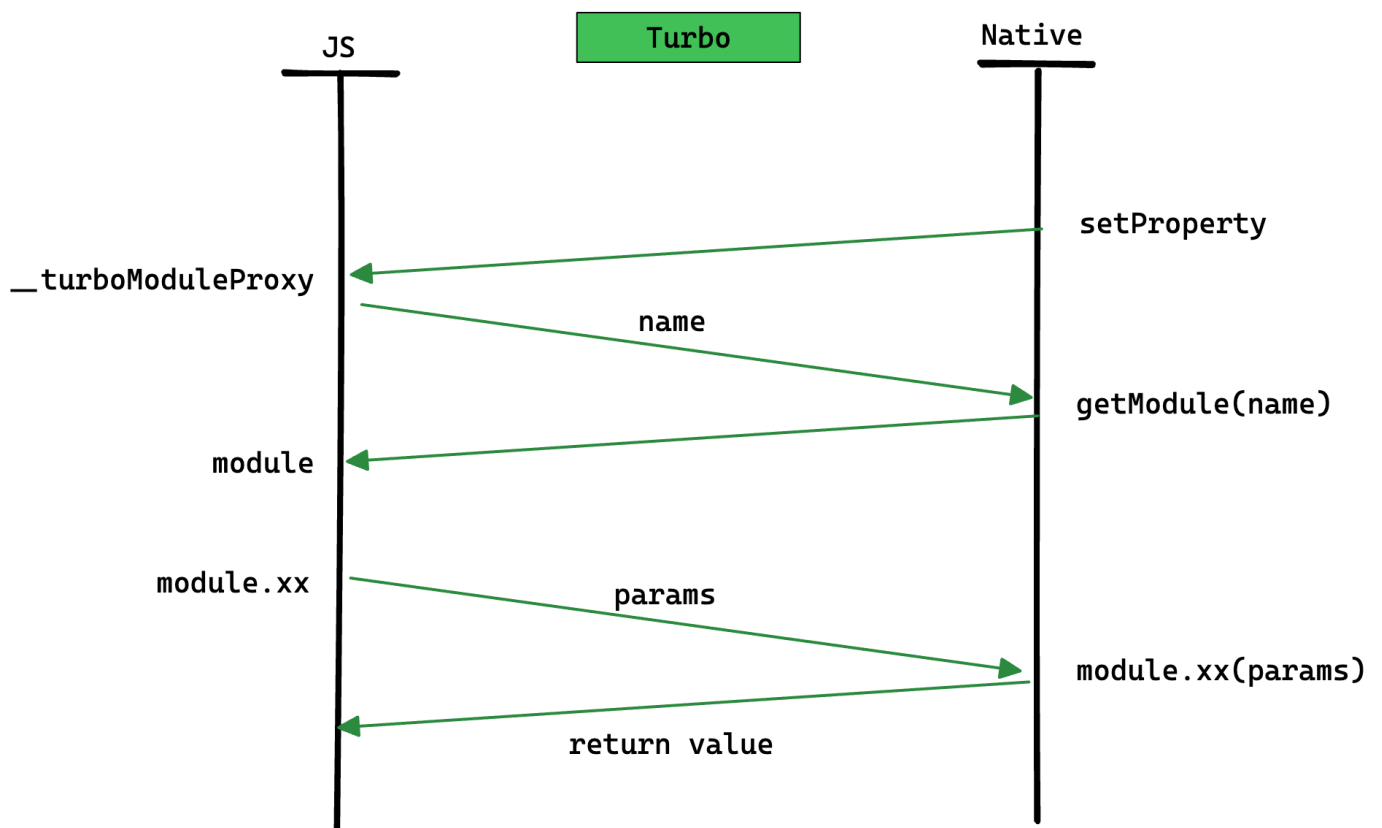
1,
[binding = TurboModuleBinding(
    std::move(moduleProvider),
    bindingMode,
    std::move(longLivedObjectCollection))](
    jsi::Runtime &rt,
    const jsi::Value &thisVal,
    const jsi::Value *args,
    size_t count) mutable {
    return binding.getModule(rt, thisVal, args, count);
}));

```

```

// Libraries/TurboModule/TurboModuleRegistry.js
// 在 js 获取 Turbo Modules
global.__turboModuleProxy(name)
// name = "Appearance"
// return = {addListener,removeListeners,getColorScheme}

```



从上述代码和时序图中，你可以看到，最开始在 C++ 文件 `TurboModuleBinding.cpp` 中，先通过 `runtime.global()` 获取了 JavaScript 全局变量 `global`，然后再通过 `setProperty` 设置了 `global` 的 `turboModuleProxy` 属性。

turboModuleProxy 属性是一个函数，函数接受一个入参是 Turbo Modules 的名字，函数的返回值是该 Turbo Module 在 Native 侧的原生实现了。比如，turboModuleProxy(“Appearance”) 的返回就是 Native 的暗黑模式 Module 生成的实例对象了，该对象包括 addListener,removeListeners,getColorScheme 方法。

对比 Bridge 和 Turbo Modules 连接点的设计，你可以看出，Bridge 是批处理，而 Turbo Modules 调用一次就是一次，调用两次就是两次。更关键的是，Bridge 任务的调用和任务的返回值是分离的，而 Turbo Modules 的调用方式和 JavaScript 系统函数的调用方式是一样的。

这就意味着，我们只需在初始化时，调用一次 turboModuleProxy(“Appearance”) 拿到 Appearance 实例就行，不用反复调用 turboModuleProxy(“Appearance”) 再次获取实例了。下次再继续调用时，直接在 JavaScript 侧拿着 Appearance.getColorScheme 调用 Native 定义的函数就可以了，这就为后续操作带来了极大的便利性。

Fabric Components

最后一个关键连接点是 Fabric Components 的连接点 nativeFabricUIManage 了，其代码如下：

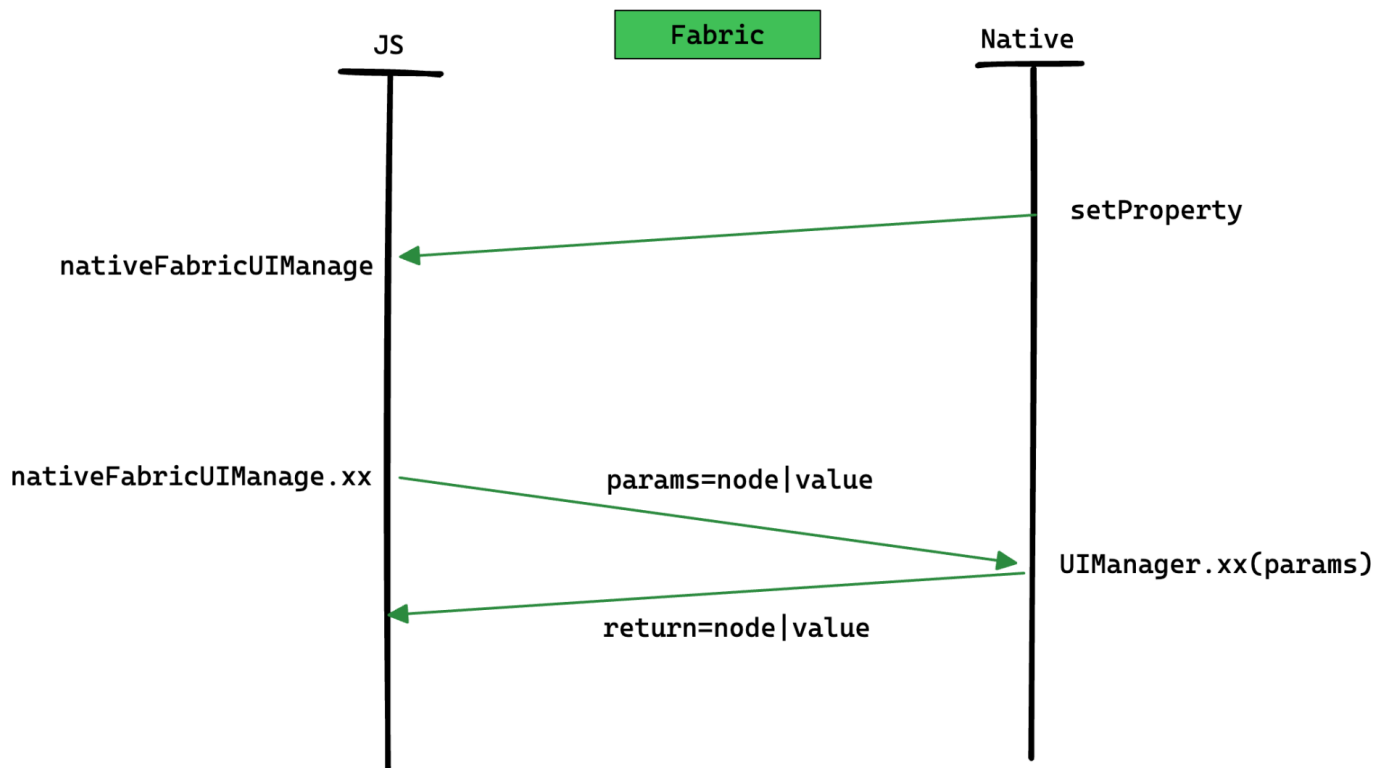
```
// ReactCommon/react/renderer/uimanager/UITManagerBinding.cpp
// 将 Native 的 uiManager 实例和运行时执行器绑定
auto uiManagerBinding =
    std::make_shared<UITManagerBinding>(uiManager, runtimeExecutor);
// 将 Native 的 uiManager 示例转换为 JS 的 Object 数据类型
auto object = js::Object::createFromHostObject(runtime, uiManagerBinding);
// 将该 object 绑定到 JS 的 global.nativeFabricUIManager 上。
runtime.global().setProperty(
    runtime, "nativeFabricUIManager", std::move(object));
}

// Libraries/Renderer/implementations/ReactFabric-dev.js
// 创建一个 Native 节点，并获取该节点的引用
const node = global.nativeFabricUIManage.createNode(
    tag, // reactTag
    viewConfig.uiViewClassName, // viewName
    rootContainerInstance, // rootTag
    updatePayload, // props
    internalInstanceHandle // internalInstanceHandle
)
```



```
// 在 JavaScript 中打印不出任何 node 的信息
console.log(node) // {}
console.log(node.__proto__) // undefined

// 但在 JavaScript 中却可以调用 Native 方法操作 node
// 向父节点 node1 中，插入子节点 node2
global.nativeFabricUIManage.appendChildNode(node1, node2)
```



JavaScript 中的 `global.nativeFabricUIManage` 是在 C++ 文件 `UIManagerBinding.cpp` 中创建的，与其对应的是 Native 的 `uiManager` 实例。

在 Native 侧，`uiManager` 实例上有操作节点的方法，比如类似 DOM 的操作方法 `createNode`、`appendChildNode`。在 JavaScript 侧，`nativeFabricUIManage` 对象上也有相对应的属性 `createNode`、`appendChildNode`，通过它们就能调用 Native 侧的实现了。

当然，React Native 属于声明式编程，开发者不会涉及对节点的具体操作，也不会直接调用 `createNode`、`appendChildNode` 去操作节点。开发者只需借助 React `setState` 控制状态即可，至于具体的节点操作会交给底层的 `ReactFabric.js` 文件进行处理。

在 `ReactFabric.js` 文件中，你可以看到 React Native 底层使用 `createNode` 创建节点的代码。`createNode` 方法的入参是 `reactTag`、`viewName`、`rootTag`、`props`、`internallInstanceHandle`。

比如，你想创建文本节点，那么你就可以将 `viewName` 的传参设置成“RCTRawText”，你想创建滚动容器节点，那么你就可以将 `viewName` 的传参设置成“RCTScrollView”。

创建完成后，Native 侧会给 JavaScript 返回一个节点的引用，虽然在 JavaScript 侧你打印不出任何属性来，该引用甚至连 `proto` 属性都没有。但是你却可以使用 `appendChildNode` 去操作这些节点，比如你可以使用 `appendChildNode` 方法把任意的 `node2` 节点插到 `node1` 节点中。

这些节点有个专有的名字叫做**影子节点 ShadowNode**，它在 JavaScript 中就如同影子一般，打印不出任何属性，但是在 Native 侧你可以断点看到它的 `props`、`state`、`children` 等属性，我给你截图下来了，你可以看看：



```
▼ facebook::react::ShadowNode
  > facebook::react::Sealable
    revision_ = (const int) 1
  > props_ = (facebook::react::Props::Share
  > children_ = (facebook::react::ShadowNode
  > state_ = (facebook::react::State::Share
    orderIndex_ = (int) 0
  > family_ = (facebook::react::ShadowNode
  > hasBeenMounted_ (std::atomic<bool>
  > traits_ (facebook::react::ShadowNodeT
```

鸟瞰图

了解完 Bridge、Turbo、Fabric 的相关实现后，你一定要在脑中过一下整个新架构的实现流程，再亲自动手去打断点，执行一下程序，看看实际执行过程是否符合你的预期。

只有当你自己真正使用 Xcode、Android Studio 到源码中断点 Bridge、Turbo、Fabric 相关代码后，你才能体会到实际的执行过程和你预期的结果有什么差别。我自己在动手断点的过程中，就大吃一惊，原来看完源码后，实际的执行过程还是和自己的预期差别很大。

最开始，我预期新架构中 Bridge 代码就是一个摆设，实际已经不会走到 Bridge 的相关执行流程了。因为官方多次强调过，Bridge 的性能是比 Turbo 和 Fabric 差的。有了 Turbo 和 Fabric 后，新架构没必要用个性能更差的 Bridge 了，是吧？

但实际执行时，我却发现新架构中居然还在调用老架构的 Bridge 实现通信。

我动手跑了一下官方提供的默认工程，发现 React Native 程序启动时，最先完成初始化的是 Bridge 部分，然后是 Turbo 部分，而 Fabric 是按需初始化的，只有 Bundle 代码调用了 Fabric 的相关逻辑时，才会初始化。

然后我又写了一些简单的创建状态视图和调用API的代码，进行了测试，又发现新架构中确实实用到了老架构的 Bridge。测试过程中发现，在 26 个 Module 中，有两个 Module 使用了 Bridge，这两个 Module 分别是 Timing 和 NativeAnimatedModule。

26 个全部 Module 我也放在了下面，就不一一念名字了，你可以自己看看，试试是否还有其他 Module 在使用 Bridge。

```
// Bridge 中 ModuleID 与 ModuleName 的对应关系：
```

```
[0] = "ViewManager"
[1] = "ActivityIndicatorViewManager"
[2] = "DatePickerManager"
[3] = "MaskedViewManager"
[4] = "ModalHostViewManager"
[5] = "ModalManager"
[6] = "ProgressViewManager"
[7] = "RefreshControlManager"
[8] = "SafeAreaViewManager"
[9] = "ScrollContentViewManager"
[10] = "ScrollViewManager"
[11] = "SegmentedControlManager"
[12] = "SliderManager"
[13] = "SwitchManager"
[14] = "UIManager"
[15] = "Timing"
[16] = "NativeAnimatedModule"
[17] = "ImageViewManager"
[18] = "BaseText"
[19] = "BaseTextInputViewManager"
[20] = "InputAccessoryViewManager"
[21] = "MultilineTextInputViewManager"
[22] = "RawText"
```

```
[23] = "SinglelineTextInputViewManager"  
[24] = "Text"  
[25] = "VirtualText"
```

我也选了几个 API 进行了测试，包括 Appearance、DeviceInfo、Platform，这些 API 又确实都使用的是 Turbo Module 进行调用的。

和 Turbo Module 相关的模块一共有 25 个，我也贴在了下面：

```
Networking  
Appearance  
FrameRateLogger  
ViewManager  
BugReporting  
RedBox  
EventDispatcher  
NativeAnimatedTurboModule  
StatusBarManager  
DeviceInfo  
HeadlessJsTaskSupport  
LogBox  
AppState  
WebSocketModule  
SourceCode  
SoundManager  
BlobModule  
KeyboardObserver  
ImageLoader  
PlatformConstants  
DevMenu  
AccessibilityManager  
DevLoadingView  
DevSettings
```

同样，我还测试了 Fabric Component，从测试结果中，可以推出 React Native 官方提供的 9 个 Component 都是通过 Fabric 进行创建和操作的，这 9 个 Component 如下：

```
SafeAreaView  
ScrollView  
View
```

Paragraph
RootView
RawText
TextInput
Text
Image

看完源码并了解 0.70 新架构预览版的现状之后，我不禁陷入了沉思。

为什么新架构预览版中明明已经大规模用上了 Turbo 和 Fabric ，还要初始化和使用 Bridge 呢？

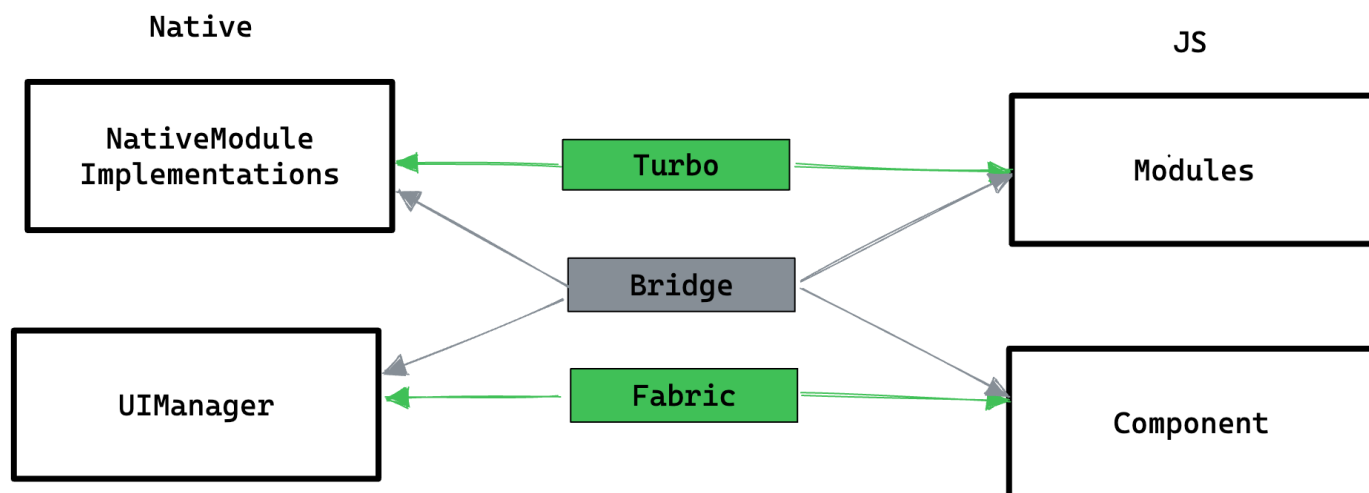
是 Bridge 代码还没来得及优化完成？又或者因为在处理 Timing、NativeAnimatedModule 这些特殊场景中，Bridge 比 Turbo 和 Fabric 性能更好呢？还是为了方便业务代码迁移，因此向下兼容了老架构呢？

我个人更倾向于最后一个答案，需要向下兼容。

一方面是因为，官方文档提到第三方库开发者需要对 Bridge、Turbo 和 Fabric 进行兼容，以方便业务代码的迁移。

另一方面是，目前基于 Bridge 的库是不能直接运行在 Turbo 和 Fabric 上的，因此从理论上推测，如果官方强制让原先所有依赖 Bridge 的业务代码、底层库都改用 Turbo 和 Fabric 进行重写，社区肯定升级不动。7 年积累的巨量代码，哪是这么容易就甩掉的包袱啊？

基于以上事实和推理，在我心目中 0.70 新架构预览版的鸟瞰图，它大概长这个样子：



新架构中，Native 和 JavaScript 通过三种方式进行通讯，分别是 Bridge、Turbo 和 Fabric，业务代码中的历史代码依旧走 Bridge 进行通讯，新代码新项目走 Turbo、Fabric 进行通信。

上述模型肯定不是最优解，而是一个过渡方案。

或许官方会在新架构稳定后，将是否使用 Bridge 进行兼容的开关暴露给开发者，由开发者进行选择，是要纯新架构的超高性能，还是要新旧架构混合的性能提升+兼容性强。

总结

今天，我们通过“读文章”、“找线头”和“鸟瞰图”三步，弄清楚了新架构的现状。

首先，在几千个文件中，快速读懂源码是有方法的。找到了 Native 与 JavaScript 连接点，就找到了读源码的突破口，这三个关键连接点分别是 `nativeFlushQueueImmediate`、`__turboModuleProxy` 和 `nativeFabricUIManager`。

通过对源码的阅读，我们可以确定，目前新架构中 Bridge、Turbo 和 Fabric 三种通讯方式是同时存在的。单从性能考量，初始化 Bridge 肯定拖慢了程序的启动耗时，但从升级成本角度出发，不一刀切地放弃所有历史代码，才会让社区更有意愿升级新架构。

思考题

你是否有跑在线上的使用老架构的 React Native 项目？你升级这些项目时最关键的 3 个考量点是什么呢？

欢迎给我留言，咱们下一讲见。

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

上一篇 28 | 追本溯源：从第一版 React Native 开始学会读懂源码

下一篇 30 | JavaScript 引擎：双向通讯底层原理是什么？

精选留言 0